

Spreadsheet 一道 LeetCode Hard 的深挖笔记

从 LC631 到工业级设计:数据模型、判环、事务、失效传播、并发的七层推演

整理于 2026-07-27 · 配套代码 fuse2.cpp

这份文档怎么用:按顺序读一遍(约 40 分钟),重点背诵第 10 章台词库与第 11 章节奏剧本;第 9 章病历簿在面试当天早上再扫一遍——那些坑全是你亲手踩出来的,看一眼就能唤醒。核心原则只有一条:**面试当场手写的只有 V1(第 3~5 章),V2 之后的所有层全部用嘴交付,面试官明确要求才动手。**

目录

- 1. 情报篇:题目出处、评分卡与成功标准
- 2. 题目规格全文与三颗隐藏雷
- 3. 第一层 · 数据模型:Cell 的身份问题
- 4. 第二层 · 解析与求值:纯函数 GetHelper
- 5. 第三层 · 判环与事务:Graph 与 Sum 的完整流程
- 6. 第四层(V2) · 缓存与失效传播:valid 位、coverage 建边
- 7. 第五层 · Range 的三档策略与 #4 权衡
- 8. 第六层 · Undo / 第七层 · 并发(讨论层)
- 9. 病历簿:五天里踩过的每一个坑
- 10. 台词库(英文,原样可用)
- 11. 60 分钟节奏剧本与考前检查清单

1. 情报篇:题目出处、评分卡与成功标准

1.1 为什么押这道题

一亩三分地上 Fuse Energy 的全部三篇面经(2025 Q2 电面、2026 Q1 电面、2026 Q2 onsite)考的是**同一道题**:简化版 Excel 计算引擎(SET / GET / SUM + 循环依赖检测, follow-up 为 UNDO)。官方 prep 文档也印证了题型: "building a lightweight in-memory system (often similar to a spreadsheet or table-like model)"。此题即 LeetCode 631 "Design Excel Sum Formula"(官方 Hard),但 Fuse 版本额外要求判环 + 事务性,难度高于原题。

尾部风险:prep 文档写了 "we're going through changes, this type of problem may vary"。应对:90% 火力打磨此题,10% 依靠已练成的通用流程(澄清规格 → 最小模型 → 纯读路径 → 写路径事务 → trace → 说权衡),题变流程不变。

1.2 官方评分卡(prepare 文档原文要点)

评分维度	对应到这道题的具体动作
------	-------------

problem-solving ability	正确的图模型、判环不变量、事务顺序
clean & maintainable code	Coordinate/Cell/Graph 分层、ForEach 抽取、纯函数 Get
how well you extend an existing solution	每次新需求走"最少改哪几行"的增量路径,不重构
clarity of reasoning & communication	写前一句方案、写后出声 trace、决策说出来("重复计数,我按出现次数累加,flag 一下")
choose appropriate data structures	hash map、图邻接表、range 压缩存储
reason about correctness when data changes	失效传播、"没有失效机制的缓存 = bug"

1.3 Top Candidate 的三个门槛(prepare 文档原话)

- **"finish with 10-15 mins to spare"** —— V1 全量约 110 行,预算 40~45 分钟,留 15 分钟进讨论层;
- **"clearly explain R/W patterns"** —— 写多读少 → lazy;读多写少 → 缓存(带失效);终极版 delta 传播;
- **"discuss multithreading / partial caching"** —— 第 8 章三档答案。

面试官:Dmitrii Zyrianov,10+ 年企业后端(Ural State Univ.,Moscow base)。评审理角是"资深后端看代码":接口、错误处理、可扩展性,不是算法竞速。旁听:Alexander Miller(recruiter,不参与技术评估)。平台 codeshare,**不运行代码**——你的"运行"就是出声 trace。

2. 题目规格全文与三颗隐藏雷

2.1 规格(按面经整理)

Set(cell, value) 将 cell 设为整数常量 value(覆盖语义)
Sum(destCell, refs) 将 destCell 设为 SUM 公式(覆盖语义)。refs 列表元素为:
单格 "B2",或矩形区域 "A1:B2"(含边界所有格)
Get(cell) 返回当前值;公式格返回实时计算结果;空格返回 0

示例:

```
Set("A1", 1)
Set("B2", 2)
Sum("D1", ["A1:B2", "B2"]) # D1 = A1+A2+B1+B2 + B2 = 5
Get("D1") # 5
Set("A1", 2)
Get("D1") # 6
```

循环依赖:若 Sum 调用会造成(直接或间接)循环依赖,该调用必须报错,且【不得修改表格状态】。

```
Sum("A1", ["A1"]) → 报错(自引用)
Sum("A1", ["B1"]); Sum("B1", ["A1"]) → 第二次调用报错
```

规模:坐标为 Excel 风格(列大写字母,支持多字母如 AA10;行为正整数),操作 1e5 级,表稀疏。Follow-up:UNDO(Set 与 Sum 均算写操作)。

2.2 三颗隐藏雷(prepare 文档警告 "You WILL miss the most critical requirement")

雷	内容	出处	拆雷动作(开场 clarify 时说)
①	重复引用按出现次数累加 :样例里 B2 在 range 里出现一次、又单独出现一次,共加两次($5 = 3 + 2$)	面经样例实锤,最可能是"那颗雷"	指着样例问:"B2 counted twice, correct?"
②	多字母列 :AA10 合法。列编码是 bijective base-26:A=1 ... Z=26, AA=27。公式必须 $col * 26 + (c - 'A' + 1)$,忘 +1 则 A 与 AA 同为 0,坐标碰撞	面经"数据规模"节实锤	写 Coordinate 时口头点出 "+1, bijective base-26"
③	稀疏表,勿物化空格 :读路径禁用 operator[] (会把空格插进 map);大 range 只遍历、不创建	由 $1e5 +$ 任意坐标推断(加分项)	用 Exist/find 守门时提一句 "avoid materializing empty cells"

另两个必须主动澄清的语义:空格 Get 返回 0(还是报错?默认 0);Sum 是覆盖语义(等价于赋值,连续两次 Sum 后者覆盖前者;Set 覆盖 Sum、Sum 覆盖 Set 同理)。

3. 第一层 · 数据模型:Cell 的身份问题

3.1 核心心智模型(本题全部正确性的地基)

一个 cell 在任何时刻只有一种身份:要么是字面量(literal),要么是一个公式(formula)。二者互斥,写操作整体覆盖。

推论 1:公式格的值不存在于任何字段里,每次 Get 时从引用现算(V1),或由带失效机制的缓存承载(V2)。

推论 2:cell 绝不是"操作列表"——不存在"一个格子累积多个历史操作、求值时依次执行"的语义(此模型曾导致三轮返工,见病历 #1)。

推论 3:将来扩展加减乘除,变的是"公式"的内部结构(表达式树),不是给 cell 挂 op 列表。

3.2 代码与自习说明

```
struct Cell {
    int val = 0; // literal 的值;公式格恒为 0(V1)或缓存值(V2)
    bool valid = false; // V1 语义:true=literal, false=formula(类型标记)
    // V2 语义扩展:公式格的 valid 兼作缓存新鲜位(见第6章)
    vector<vector<Coordinate>> depends; // 公式的引用列表:
    // size==1 → 单格引用 [co]
    // size==2 → 矩形 [左上, 右下](压缩存储,不展开)
    Cell() {} // 默认构造 = 空公式壳(由 Sum 填充)
    Cell(int val) { this->val = val; this->valid = true; } // literal 构造
};
```

自习说明:

- `val = 0` 的默认初始化不可省——曾因未初始化的 `int val` 在 `operator[]` 默认构造路径上被加进总和,产生"取决于内存垃圾"的 UB(病历 #3);

- depends 用"压缩 range"(只存左上/右下两个角)而不是展开成格子列表——这是第 7 章内存策略的第一块基石;
- 面试口径:如果说得更现代,提一句 "production 里我会用 `std::variant<int, Formula>` 显式互斥;这里用 `bool` 标记保持简单"——一句话拿到分,不必真写 `variant`。

4. 第二层 · 解析与求值:纯函数 GetHelper

4.1 Coordinate:解析与编码

```
struct Coordinate {
    int row, col;
    string name; // "row:col", 作图的 key(选字符串是为了调试可见)
    Coordinate(const string & cell) { // 解析 "AA10" → col=27, row=10
        size_t i = 0; row = 0; col = 0;
        while (i < cell.size()) {
            if (cell[i] >= 'A' && cell[i] <= 'Z')
                col = col * 26 + (cell[i] - 'A' + 1); // ← +1: bijective base-26(雷②)
            else if (cell[i] >= '0' && cell[i] <= '9')
                row = row * 10 + cell[i] - '0';
            i++;
        }
        name = to_string(row) + ":" + to_string(col);
    }
    Coordinate(int row, int col) { ... name 同样生成 ... } // range 展开时用
};
```

自习说明:解析器对非法字符是"静默跳过"——顺带能过 "A1:B2" 里的冒号,但也会静默接受 "a1"、"1A" 等垃圾输入。面试时主动说一句 "the parser silently skips unknown chars — in production I'd validate and throw" 即可,不用写校验。name 用 "row:col" 而不是原始 "AA10":避免为 range 展开的格子写"列号→字母"的反向拼接(base-26 逆运算约 10 行)。曾因 `Coordinate(int, int)` 忘生成 name,导致 range 展开的格子在图里全叫空串、互相碰撞(病历 #7)。

4.2 ForEachDependencyCell:遍历骨架的抽取

```
template <typename Func>
void ForEachDependencyCell(const vector<Coordinate> & dep, Func && func) {
    if (dep.size() == 1) { func(dep[0]); return; } // 单格
    for (int r = dep[0].row; r <= dep[1].row; r++) // 矩形:行×列双层循环
        for (int c = dep[0].col; c <= dep[1].col; c++)
            func(Coordinate(r, c));
}
```

自习说明:"单格 or 矩形展开"的骨架在 Get / 删边 / 加边三处重复出现,第三次出现时抽取——**抽取时机比抽取本身重要**:第一遍就写它 = 过早抽象,模式出现三次后抽 = refactoring instinct(面试中当场说 "I notice I'm repeating the expansion — let me extract it" 是强信号)。模板参数 + 转发引用让 lambda 零开销内联;需要中途 break 的调用方(加边判环)用外部 flag + 早退 return 解决。曾用范围 for `for (auto dep : ...)` 每轮拷贝整个 vector——改 `const auto &` (病历 #8)。

4.3 Exist 与 GetHelper:纯函数求值

```
bool Exist(const Coordinate & co) {                // 稀疏守门:只查不插(雷③)
    if (_cache.count(co.row) == 0) return false;
    if (_cache[co.row].count(co.col) == 0) return false;
    return true;
}

int GetHelper(const Coordinate & target_co) {        // ★ 纯函数:只返回,不写任何状态(V1)
    if (!Exist(target_co)) return 0;                // 空格 → 0(已澄清的默认)
    Cell & cell = _cache[target_co.row][target_co.col];
    if (cell.valid) return cell.val;                // literal:0(1) 直读
    int val = 0;                                    // formula:现算
    for (const auto & dep : cell.depends)
        ForEachDependencyCell(dep, [&](const Coordinate & co) {
            if (Exist(co)) val += GetHelper(co);    // 递归;重复引用自然按次数累加(雷①)
        });
    return val;
}
```

军规(用三版 bug 换来的):V1 的 Get 家族里,赋值号左边不允许出现 _cache。

违反此规的三次事故:① 把算出的和写回公式格 val → 二次读翻倍(5→10);② 条件写回 literal → 无收益纯风险;③ 手误把累加器写进子格 → 读操作销毁用户数据。写回 = 无失效机制的缓存 = bug。真缓存是 V2 的独立工程(带失效),不是求值时"顺手存一下"。

复杂度口径:Set O(1);Get O(依赖树规模)——这就是 lazy 的定价:写便宜、读付费。写多读少场景它就是终态;读多写少 → 引出 V2。这句权衡在写 Sum 前主动说(评分卡 R/W patterns 项)。

5. 第三层 · 判环与事务:Graph 与 Sum 的完整流程

5.1 图的定义(一条注释钉死方向)

```
// EDGE DIRECTION: dep -> target (被依赖的格子 指向 使用它的公式格)
// 即"A1 变化会影响谁"。此方向同时服务:判环(本章)与失效传播(第6章,同一张图两用)
struct Graph {
    unordered_map<string, unordered_set<string>> data;    // 邻接表;set 天然去重
    void addEdges(const string & from, const string & to) { data[from].insert(to); }
    void removeEdges(const string & from, const string & to) {
        if (data.count(from) == 0) return;
        data[from].erase(to);    // 曾误写 data.erase(to):把 to 的整个邻接表删了(病历 #9)
    }
}
```

自习说明——方向的历史:最初实现是 target→ref("我引用了谁"),判环从新 ref 顺走找 target;后为让同一张图直接服务失效传播,整体翻转为 dep→target("谁受我影响"),判环改为从 target 顺走找新 ref 的镜像问法。**两个方向互为镜像、判定等价,唯一纪律是全程一致**——口头定义与代码实现曾两次相反(病历 #10),解药就是顶上那行注释,写码前先敲它。

5.2 判环:AddEdgeHasCycle

```
// 语义:若加入新边 from→to,是否成环?
// 不变量:调用前图无环(每次 Sum 都守住它)⇒ 新环必含新边 ⇒
//      只需问:【从 to 出发沿现有边,能否走回 from?】一次 DFS,0(可达子图)
bool AddEdgeHasCycle(const string & from, const string & to) {
    if (from == to) return true; // 自引用:图里还没有边,DFS 查不到,
                                // 必须特判(病历 #11,面经点名用例)
    unordered_set<string> visited; // 防重走。注意:visited ≠ 环!
    visited.insert(to); // diamond(菱形共享依赖)重访是合法的
    function<bool(const string &)> dfs = [&](const string & r) -> bool {
        if (data.count(r) == 0) return false;
        for (auto & node : data[r]) {
            if (node == from) return true; // 走回起点 → 环
            if (visited.count(node) == 0) {
                visited.insert(node);
                if (dfs(node)) return true;
            }
        }
        return false;
    };
    return dfs(to);
}
```

自习说明——为什么不用拓扑排序:拓扑排序验证"整张图无环",O(全图),还要维护入度;这里靠不变量只需验证"新环必过新边"这一件事,DFS O(可达子图),约 12 行零额外状态。结论相同,DFS 严格更便宜——"最小工具"原则的标准案例。另:"visited 过 = 有环"是错误判据(病历 #2)——菱形 D1→{B1,C1}→A1 里 A1 被访问两次,无环且其值合法地累加两次;环的唯一判据是**回到起点**。

5.3 Sum 的事务流程(你选定的口径:先删后检、失败恢复)

```
void Sum(const string & target_cell, vector<string> & cells) {
    Coordinate co(target_cell);
    // —— ① 备份(为失败恢复) ——
    Cell old_cell; bool has_content = false;
    if (Exist(co)) { has_content = true; old_cell = _cache[co.row][co.col];
                    RemoveEdgeHelper(co); } // 删掉 target 自己的旧出边(覆盖语义)
    // —— ② 解析 refs → 新公式(压缩 range) ——
    Cell nc;
    for (auto & cell : cells) {
        size_t p = cell.find(':');
        vector<Coordinate> dep;
        if (p == string::npos) dep.push_back(Coordinate(cell));
        else { dep.push_back(Coordinate(cell.substr(0, p)));
                dep.push_back(Coordinate(cell.substr(p + 1))); } // p+1:跳过冒号
        nc.depends.push_back(dep);
    }
    _cache[co.row][co.col] = nc; // ③ 落盘新公式
    // —— ④ 逐条检环并加边(addEdgeHelper 内部:检出环即停,
    //      并回滚本次已加的新边 added_edges) ——
    bool has_cycle = false;
    addEdgeHelper(co, has_cycle);
    // —— ⑤ 失败分支:恢复到调用前状态 + 报错 ——
    if (has_cycle) {
        if (has_content) { // 5a 原有内容:整格恢复 + 重建旧边
            _cache[co.row][co.col] = old_cell;
        }
    }
}
```

```

        bool dummy = false; addEdgeHelper(co, dummy);
    } else {
        _cache[co.row].erase(co.col);
    }
    throw runtime_error("cycle detected");
}
}

```

自习说明——事务性的两种合法口径:规格原文"报错且不应修改表格状态"。(a) 先检后删:检环时旧边虽在,但DFS碰到target即终止、不会展开target的旧出边,判定不受影响,失败零回滚;(b) 先删后检(当前实现):直觉上更"干净的图",代价是失败分支必须恢复(备份旧Cell + 重建旧边 + 新边回滚)。两者判定等价、都正确——**面试时选哪个都行,关键是失败分支闭环 + throw**。还有第三个送分动作:写之前问一句 "on cycle, should the sheet remain unchanged, or is fail-fast acceptable?"——十秒钟的规格意识。

5.4 V1 交付物清单(周二手写的全部)

组件	行数≈	要点
Coordinate(解析+name)	25	+1 编码;name 生成
Cell / SpreadSheet 骨架	15	literal XOR formula
Exist + GetHelper + ForEach	30	纯函数;不物化
Graph(add/remove/判环)	30	自引用特判;方向注释
Set / Get / Sum(事务)	35	备份-删-写-检-恢复-throw
main 断言(样例+自引用+两步环+覆盖)	15	出声 trace 替代运行

合计约 150 行(含空行),预算 40~45 分钟。**断言用例固定四组:**官方样例(5→6)、Sum("B3",["B3"]) 抛错、两步环 A1→B1→A1 第二次抛错且 Get 仍正常、Set 覆盖公式/ Sum 覆盖常量各一条。

6. 第四层(V2) · 缓存与失效传播

本章全部内容为口头交付层,面试官明确要求实现且时间富余才动手。代码已在 fuse2.cpp 落地并经编译 + 断言验证。

6.1 动机与架构(一句话版)

lazy 的重复 Get 会反复重扫依赖树 → 给公式格加缓存位;**缓存的灵魂是失效**:Set/Sum 改变一个格子后,沿"谁受我影响"的方向把下游公式全部标脏 → **判环建的 dep→target 图**,方向恰好就是失效要走的方向——**同一张图两用**。Get 命中新鲜缓存 O(1),脏了才重算并回填。

6.2 缓存位的语义扩展

```

// V1: valid ⇔ "是 literal"(类型标记)
// V2: valid 对公式格兼作"缓存新鲜位":
//   literal:valid 恒 true,val 即值
//   formula:valid=true → val 是新鲜缓存;valid=false → 脏,需重算

```



```
int GetHelper(const Coordinate & target_co) {
    ...
    if (cell.valid) return cell.val;          // 命中(literal 或新鲜缓存)
    int val = 0;
    ... 递归求和 ...
    cell.val = val; cell.valid = true;        // ★ V2 里这两行是【设计出来的回填】：
    return val;                               // 合法性来自下方失效机制的存在。
}                                              // 没有失效机制时,同样两行 = bug(军规)
```

自习说明:更干净的设计是独立 `bool dirty` (类型标记与状态标记分离,literal/formula × 新鲜/脏 四象限不挤压)——口头提这一句,当前复用 `valid` 的实现在"literal 恒 true"的约定下也自治。**同一段"算完写回"的代码,V1 里是三次事故的元凶、V2 里是正确设计**——分界线只有一条:失效机制是否存在。这句对比是你被追问缓存时最有质感的答案。

6.3 失效传播:InvalidateDownstream

```
void InvalidateDownstream(const string & source) {
    unordered_set<string> vis; queue<string> q;
    vis.insert(source); q.push(source);
    while (!q.empty()) {                          // BFS 沿 dep→target 边走
        string cur = q.front(); q.pop();
        if (_graph.data.count(cur) == 0) continue;
        for (const auto & nxt : _graph.data[cur]) {
            if (!vis.insert(nxt).second) continue; // 防重入(diamond)
            q.push(nxt);
            if (_sum_ranges.count(nxt)) {          // 只有公式格需要标脏
                Coordinate co(0, 0);
                if (ParseName(nxt, co) && Exist(co))
                    _cache[co.row][co.col].valid = false;
            }
        }
    }
}
// 调用点:Set 落盘后、Sum 成功落盘后(公式自身也标脏:_cache[..].valid=false)
```

自习说明:传播是传递的——A1 变,依赖 A1 的 D1 脏,依赖 D1 的 F1 也脏,BFS 一次走完整个下游闭包。只标脏、不重算(重算延迟到下次 Get)——这是"标脏"与"eager 重算"的分界;eager 版还要按拓扑序更新,复杂一档,口头对比即可。终极优化 delta 传播:Set 时计算 $\text{delta} = \text{新值} - \text{旧值}$,下游按引用重数 $+= \text{重数} \times \text{delta}$,连重算都省——说出它即锁定 strong signal。

6.4 核心洞:coverage 建边(本项目最后一个重大 bug)

问题:建边若以 `Exist`(格子已 set)为前提,"先 Sum 引用、后物化"的格子会漏边。两种发作:

- ① 漏环崩溃: `Sum("D1", ["A1"])` (A1 不存在,无边) → `Sum("A1", ["D1"])` → 图查不到环 → 真环落盘 → Get 无限递归;
- ② 无声算错(更阴险): `Sum("F1", ["F2"])` → 后 `Sum("F2", ["A1"])` 时 `F2→F1` 边缺失 → 此后 `Set("A1")` 失效传播到 F2 断链 → `Get("F1")` 返回 stale 缓存。

根因一句话:图存在性 ≠ 值存在性。依赖关系由公式声明,与格子当下有没有值无关。

```
// 修复:任何"物化"一个格子的操作(Set 与 Sum 都会!),都要让新格子
// 主动登记"我落在哪些已有公式的 range 里",并补边 新格子 → 那些公式
```



```

void AttachCoverageEdges(const Coordinate & co, bool check_cycle,
                        bool & has_cycle, vector<string> & attached_targets) {
    for (const auto & [target, ranges] : _sum_ranges) { // 遍历所有公式的登记区间
        if (target == co.name) continue;
        bool covered = false;
        for (const auto & rg : ranges)
            if (InRange(co, rg.start, rg.end)) { covered = true; break; }
        if (!covered) continue;
        if (check_cycle && _graph.AddEdgeHasCycle(co.name, target)) {
            has_cycle = true; return; // Sum 路径参与判环
        }
        _graph.addEdges(co.name, target);
        attached_targets.push_back(target); // 记录,供失败回滚
    }
}
// _sum_ranges: unordered_map<string, vector<Range>> —— 每个公式格登记其压缩区间
// Set 调用(不判环:Literal 不可能成环);Sum 落盘后调用(判环,失败回滚 attached)

```

自习说明:回归断言两组已入 main(上述两个复现场景),编译运行全过。此洞的发现-修复完整走了一遍"诊断(自己)→ 补丁(GPT 执行)→ 回归断言 → 机器验证"的工业流程——面试里如果聊到 testing,这个故事本身就是答案。

7. 第五层 · Range 的三档策略与 #4 权衡

7.1 Range 内存三档(被问 "range 很大怎么办" 时的完整答案)

档	做法	成本	结论
全展开	range 按格拆成边	A1:ZZ9999 → 数百万边,内存爆	不可取
压缩(当前)	只存左上/右下;失效/判环时做"点∈矩形"判断	每公式 O(区间数);Set 线性扫公式的区间表	1e5 规模的正解
区间索引	interval tree 按点查覆盖区间	O(log n + k) 查询;实现复杂一档	range 数巨大才升级;"我需要真实竞争数据才会走到这步"

7.2 "未存在格子"的两档 cost(#4 的最终答案,两档都是自己推的)

方案	机制	付出	换得
占位键入图	Sum 时不管格子存不存在,边照加(字符串键而已)	每条公式几个哈希键的内存	Set 时 O(1) 精确查 data[name]
扫公式区间表(当前)	物化时刻(Set/Sum)扫 _sum_ranges 判覆盖、补边	每次物化 O(公式数×区间数)	零占位内存;公式格稀疏时极便宜

选择依据一句话:**Set 频率 vs 公式密度**——写密集选占位键,公式稀疏选扫描。这正是评分卡 "R/W patterns" 的第二个用武之地。

8. 第六层 · Undo / 第七层 · 并发(讨论层)

8.1 Undo(面经明示的 follow-up;讨论为主,富余才写)

- **与事务回滚的区别:**Sum 内部的备份恢复是"失败时的回滚",调用结束即弃;Undo 是"撤销上一次成功的操作",要求历史留存。机制相似,生命周期不同。
- **设计:**历史栈,每次成功写操作 push (坐标,完整旧 Cell 状态,是否存在过)。必须存完整状态而非旧值——旧状态可能是公式,只存 int 会把公式退化成常量(1p3a 参考答案就有此 bug,指出它即超车)。Undo = 弹栈 → 整格恢复(不存在则 erase)→ 图同步(RemoveEdgeHelper / addEdgeHelper / AttachCoverageEdges 原样复用)→ 失效传播。
- **主动澄清两点:**失败的 Sum 不入栈(未改状态);栈空时 no-op 还是报错(自定并 flag)。追问 Redo:双栈,新写操作清空 redo 栈(标准编辑器语义)。

8.2 并发三档(prepare 文档点名 "if possible, discuss")

定调句(背诵):"锁的粒度应该跟随操作的作用域——这里 Get 读整棵依赖树、Sum 检环读全图,操作是图作用域的,所以我从一把读写锁开始。"

1. **shared_mutex 打底:**Get 共享锁,Set/Sum 独占锁;判环 + 落盘 + 建边天然同临界区,原子性免费。读多写少已经很能打。**V2 注意:Get 会回填缓存 = 写操作**,共享锁下要么升级锁、要么缓存字段用 atomic/单独小锁——被追问时说出这一层是真加分。
2. **读极多:**snapshot / copy-on-write——写时构建新版本原子替换,读无锁读旧版。
3. **细粒度:**分片锁(shard by region)+ 跨片按序取锁防死锁;"复杂度涨一个量级,要看到真实竞争数据才走到这一步"。

cell 级锁为什么不对:不只是 map rehash 的工程难,根本在语义——操作天然跨格,锁单格防不住"我在算 D1 的和,你同时改了 A1"。**lock-free 为什么不用:**CAS/无锁结构适合固定小临界区,这里写操作作用域是动态子图;"我做过 lock-free,但这里不匹配"——亮兵器库 + 不乱拔刀,一句顶十句。

9. 病历簿:五天里踩过的每一个坑

复习价值最高的一章。每条都是亲手踩的,面试当天早上扫一遍。

#	坑	现象	根因/教训
1	cell = 操作列表模型	sum 语义修了三轮:旧 val 残留、两次 sum 叠加、"get 完删 ops"连样例都过不了	cell 是身份(literal XOR formula),不是操作日志。为想象中的加减乘除预铺 op 框架 = 过早抽象,未来到来时照样重写
2	"visited = 环"	diamond 共享依赖被误判成环	环的唯一判据是回到起点;visited 只防重走
3	未初始化 int val	operator[] 默认构造出垃圾值被加进总和,"有时对"	结构体成员给默认值;UB 是老后端一眼盯的地方
4	无失效缓存(三连)	①写回公式格 → 二次读 5→10;②条件写回 → 零收益;③手误写子格 → 读毁数据	军规:V1 读路径禁写 _cache。缓存必须与失效机制同生共死
5	读路径 operator[] 物化	读一次大 range,百万空格进 map,稀疏变稠密	读路径用 count/find;写路径才允许 operator[]
6	列编码忘 +1	A 与 AA 同为 0,坐标碰撞	bijective base-26:A=1;写 Coordinate 时口头自检
7	Coordinate(int,int) 忘 name	range 展开格在图里全叫 ""互撞	两个构造函数保持不变式一致;或用坐标做 key 让问题消失
8	for (auto dep : ...) 拷贝	每轮深拷 vector	const auto&;三处签名 const string& 同理
9	data.erase(to) 删整表	removeEdges 把对方整个邻接表清了	data[from].erase(to);容器 API 慢半拍时先想清层级
10	口头方向 ≠ 代码方向 (两次)	嘴上 ref→target、手写 target→ref,自我怀疑"建反了"	两方向互为镜像皆对,纪律是一致;写码前先敲方向注释
11	自引用漏检	Sum("B3",["B3"]) 无边可走,DFS 返回 false → Get 栈溢出	from==to 特判置顶;这是面经点名用例
12	新格检出环不清理	回滚条件 has_content&& 把"格子本不存在"分支漏了,坏公式滞留 → Get 无限递归	失败清理两种情况:恢复旧的、删掉新的;只写一半 = 没写
13	coverage 只在 Set 做	Sum 物化的格子漏登记 → 漏环崩溃 / stale 静默错值	图存在性 ≠ 值存在性;所有物化点(Set 和 Sum)统一走 AttachCoverageEdges
14	throw 迟迟不落	检出环静默返回,调用方无感知	"报错"是规格验收项,不是风格项
15	测试两次被省略 (Harvey 场)	"好写,节省时间" / del 功能零断言	写完函数手指自动去 main 补 assert;a feature without a test didn't happen

9.1 三个元模式(比单个坑更重要)

1. **先写后辩:naive** 先落地拿正确性,优化只做预告。"没想清楚"的解药不是继续想,是动笔——表达(代码本身就是思考器官;"想清楚才能写"的完美时刻结构上不存在,是写让它出现。
2. **只写被要求的:**每个字段/每行 if 必须能指着说出对应哪条需求;指不出的是赌注。手痒的优化说出来存档,不写进代码(泄压阀台词见第 10 章)。
3. **每段写完出声 trace:**no-execution 环境里 trace 就是运行。typo(= val vs = sub_val)、复制粘贴(dep[0] vs co)这类编译器抓不到的错,全靠这一步当场击毙。

10. 台词库(中英对照:英文原样可用,中文帮助内化)

10.1 开场澄清(前 3-5 分钟)

"Before I code, let me confirm a few things: empty cells return 0 on Get? In the example, B2 appears in the range and again as a single ref — so it's counted twice, per occurrence? And Sum overwrites whatever the cell held before — assignment semantics? Columns can be multi-letter like AA10? On cycle detection — should the sheet remain unchanged, or is fail-fast acceptable?"

中文:"写码之前确认几件事:空格子 Get 返回 0 吗?样例里 B2 在 range 里出现一次、又单独出现一次——按出现次数算两次,对吗?Sum 会覆盖格子原有内容——赋值语义?列可以是多字母的,比如 AA10?检出循环依赖时——表格必须保持不变,还是 fail-fast 可以接受?"

10.2 方案陈述与 R/W 权衡(写码前 1 分钟)

"I'll start with the straightforward lazy version to get correctness: a cell is either a literal or a formula, Get evaluates formulas recursively and is a pure function. Set stays $O(1)$, Get pays $O(\text{dependency tree})$ — good for write-heavy. If reads dominate, I already see two optimizations — dirty-flag caching with invalidation along the dependency graph, and delta propagation — happy to go there once this works."

中文:"我先写直白的 lazy 版拿正确性:cell 要么是字面量要么是公式,Get 递归求值且是纯函数。Set 保持 $O(1)$,Get 付出 $O(\text{依赖树})$ ——适合写多读少。如果读占主导,我已经看到两个优化方向——dirty 位缓存配合沿依赖图的失效传播,以及 delta 传播——基础版跑对之后随时可以去做。"

10.3 泄压阀(手痒想优化时)

"Here I could add caching / delta propagation — I'll note it and come back if performance becomes a requirement."

中文:"这里可以加缓存 / delta 传播——我先记下来,等性能成为需求时再回来做。"(把优化欲望说出来存档,代码保持干净——这是"想到就想写"冲动的唯一出口)

10.4 判环方案(写 Graph 前)

"Edges point from a cell to the formulas that depend on it — same direction invalidation will need later, so one graph serves both. Invariant: the sheet is acyclic before each Sum, so any new cycle must pass through the new edge — one DFS asking 'can I walk from the target back to a new ref' settles it. No topological sort needed. Per-Sum DFS is the minimum cost of the correctness guarantee; the real optimization is graph locality, not avoiding the check."

中文:"边从格子指向依赖它的公式——之后失效传播需要的正是同一个方向,所以一张图两用。不变量:每次 Sum 之前表格无环,所以任何新环必然经过新边——一次 DFS 问'从 target 能否走到某个新 ref'就解决了,不需要拓扑排序。每次 Sum 做一次 DFS 是正确性保证的最低成本;真正的优化方向是图的局部性,而不是省掉这个检查。"

10.5 重构时机(第三次重复出现时)

"I notice I'm repeating the range-expansion logic — let me extract a ForEachDependencyCell helper."

中文:"我注意到 range 展开的逻辑在重复了——抽一个 ForEachDependencyCell helper 出来。"(必须在第三次重复之后说;第一遍就抽 = 过早抽象 + 背题嫌疑)

10.6 缓存与失效(被追问 repeated gets)

"Formula cells get a cached value plus a dirty bit. The soul of the cache is invalidation: on Set, I BFS along the dependency graph and mark downstream formulas dirty — a cache without invalidation isn't an optimization, it's a bug. The ultimate version is delta propagation: on Set, push $(\text{new} - \text{old}) \times \text{multiplicity}$ to dependents — no recomputation at all."

中文:"公式格加一个缓存值和一个 dirty 位。缓存的灵魂是失效:Set 时沿依赖图 BFS,把下游公式全部标脏——没有失效机制的缓存不是优化,是 bug。终极版本是 delta 传播:Set 时把 $(\text{新值} - \text{旧值}) \times \text{引用重数}$ 直接推给依赖者——连重算都省了。"

10.7 未存在格子与 range(被追问 #4 / 内存)

"Dependency existence isn't value existence — a formula's edges are declared by the formula, not by whether the cell has a value yet. Two ways to pay for it: placeholder keys in the graph ($O(1)$ lookup on Set, a few hash keys of memory per formula), or scanning registered formula ranges at materialization time (zero extra memory, $O(\text{formulas})$ per Set). Which wins depends on Set frequency versus formula density. Ranges I never expand — I store corners and do point-in-rectangle checks; with many ranges you'd upgrade to an interval tree, but at $1e5$ operations a linear scan is fine and I wouldn't add it prematurely."

中文:"依赖关系的存在 $\neq$ 值的存在——公式的边由公式声明,与格子当下有没有值无关。两种付费方式:图里放占位键(Set 时 $O(1)$ 查询,每条公式付几个哈希键的内存),或在物化时刻扫描已登记的公式区间表(零占位内存,每次 Set 付 $O(\text{公式数})$ 的查询)。哪个赢取决于 Set 频率对公式密度。Range 我永远不展开——只存两个角做点在矩形内的判断;range 极多时可以升级 interval tree,但 $1e5$ 操作规模下线性扫描就够,我不会提前上它。"

10.8 并发(收官)

"Lock granularity should follow operation scope — Get reads a dependency tree, Sum's cycle check reads the whole graph, so these are graph-scoped operations and I'd start with a single reader-writer lock: shared for Get, exclusive for Set/Sum, which makes check-and-commit atomic for free. Read-heavy $\rightarrow$ copy-on-write snapshots. I've built lock-free structures before, but they fit fixed small critical sections — here the write scope is a dynamic subgraph, so lock-free is the wrong tool."

中文:"锁的粒度应该跟随操作的作用域——Get 要读整棵依赖树,Sum 的判环要读全图,这些是图作用域的操作,所以我会从一把读写锁开始:Get 共享锁、Set/Sum 独占锁,检环+落盘天然就是原子的。读极多的场景升级 copy-on-write 快照。我以前写过 lock-free 结构,但它适合固定的小临界区——这里写操作的作用域是动态子图,lock-free 是错误的工具。"

10.9 经验收官句(聊到 testing / 设计哲学时)

"My rule on this problem: get the relationships right first — edges, rollback, invalidation — then talk performance. And every feature gets an assert the moment it's written; in a no-execution editor, tracing the example aloud is my test runner."

中文:"我在这道题上的规矩:先把关系做对——建边、回滚、失效——再谈性能。每个功能写完的那一刻就补 assert;在不能运行代码的编辑器里,出声 trace 样例就是我的测试运行器。"

11. 60 分钟节奏剧本与考前检查清单

11.1 节奏剧本

时间	动作	产出
0-5'	澄清(台词 10.1)+ 方案陈述(10.2)+ 敲下方向注释	规格对齐,面试官点头 = 动笔信号(不等内心完美感)
5-20'	Stage 1+2:Coordinate → Cell → Exist → GetHelper → Set/Sum(无判环)	出声 trace 官方样例:5 → 6
20-35'	Stage 3:Graph + AddEdgeHasCycle(自引用特判)+ Sum 事务化 + throw	trace 自引用与两步环;main 补四组断言
35-45'	收尾:const 引用过一遍;ForEach 抽取(富余才做,或用台词 10.5 提一句)	"core solution is solid" 判定到手
45-58'	讨论层,跟着追问逐层放:undo → 缓存/失效 → delta → #4 两档 → range 三档 → 并发三档	top candidate 信号
58-60'	反问(备两个):Vault...不,这是 Fuse——问 telemetry/trading 侧工程挑战;或 "这套引擎在你们哪个真实系统里有影子?"	兴趣信号

11.2 全程四条硬规则(机械执行)

1. 先写 API 签名和 struct,对着需求逐条问"这条哪个字段接",接不上的删;
2. 每个字段/每行 if 能指出对应需求,指不出 = 赌注,不写;
3. 新需求来了先说"这需要改 X 函数的 Y 处"再动手;
4. 手痒的优化用台词 10.3 说出来存档,不写。

11.3 考前最后一眼(当天早上)

- 列编码 +1;name 两处构造都生成;读路径无 operator[]; const string&;
- Get 家族赋值号左边无 _cache(V1);
- Sum 顺序:备份 → 删旧边 → 落盘 → 检环加边 → (失败:恢复/erase + throw);
- 自引用 from==to 特判;visited ≠ 环;
- B2 计两次;空格 = 0;Sum 覆盖;
- 写完每段:出声 trace;写完每个功能:main 里 assert;
- VSCode 关补全练过手感;时区:6pm GMT+1 = 美东 1pm,提前 15 分钟进会;
- 心态锚点:第一遍是"想清楚的草稿",不是成品;慢而稳赢过快而崩。

一句话总纲:V1 手上功夫(45 分钟,四组断言,出声 trace),V2 以后全是嘴上弹药(每一句背后都有你亲手踩过的坑)。这道题你已经从规格推到并发的负空间——周二房间里,你是最懂它的人。